Anchor — Project Proposal
Neurodivergent Task Management & Focus System
NCI Higher Diploma in Computing | H8HDP | HDAIML_SEP25OL / HDSDEV_SEP25

Project Name	Anchor — Neurodivergent Productivity & Focus Web Application
Student Name	Jacob Benarros Neto
Student Number	X20208731
Module / Cohort	HDAIML_SEP25OL / HDSDEV_SEP25 — Project starting in May 2026
Date	8 June 2026
Referencing Style	IEEE

1. Objectives and Problem Statement

Adults with **Attention Deficit Hyperactivity Disorder (ADHD)** face a specific, well-documented cognitive problem that generic productivity software ignores entirely: they cannot reliably perceive time. Barkley [1] calls this *time blindness*, a neurological failure to sense duration. Around 50% of ADHD adults also have **Generalised Anxiety Disorder (GAD)** [2], which adds to the problem.

Existing tools do not solve this. Todoist and Notion are built for neurotypical users, they require the user to self-regulate time, which is precisely the capacity that ADHD impairs. Goblin.tools [3] uses AI for task decomposition but stores nothing; it has no accounts, no persistence, and no API for integration. Tiimo (iOS, €70/year) provides visual scheduling but overwhelming features and UX. General AI assistants like ChatGPT require manual prompt construction every session, added cognitive load that defeats the purpose for ADHD users.

Anchor addresses three failures simultaneously: task initiation paralysis, time blindness, and decision fatigue. It does this through a hybrid architecture, a deterministic priority algorithm that sorts tasks with zero AI cost, a live ADHD, calibrated countdown timer that auto-starts from the task's time estimate, and a Gemini-powered AI breakdown engine reserved for the one problem that genuinely requires natural language reasoning: decomposing a vague,

overwhelming task into concrete physical steps.

---

2. Background and Related Work

Barkley [1] identifies external scaffolding as the strongest evidence-based ADHD intervention. The brain cannot self-regulate, so the environment must do it instead. This is the core design principle behind Anchor's timer: it imposes external time structure without asking the user to initiate it.

Gollwitzer [4] demonstrates that *implementation intentions* , specific "when-then" plans for action, increase task follow-through by up to 300% compared to vague intention-setting. This directly informs Anchor's AI breakdown engine: instead of telling users *what* to do, it tells them *exactly which physical action to take first*, e.g. “Open [file]. Type one sentence."

No existing web application combines all three of the following:

1. A **persistent task management system** with algorithmic priority sorting calibrated to ADHD-specific factors (urgency decay, user energy, importance)
2. An **auto-starting visual countdown timer** with three clinically-grounded state transitions designed around ADHD attention patterns
3. A **Gemini AI breakdown engine** using structured JSON output to decompose tasks into physically-actionable steps, with steps revealed one at a time to prevent overwhelm

The gap is real, the market exists, and the clinical research supports the design choices. Anchor is not a variation of an existing tool, it is a different category of product.

---

3. Proposed Solution

Anchor delivers four core features. Each solves a specific, named cognitive problem. Features are sequenced so that the application is fully functional and demonstrable without the AI layer, Gemini is an enhancement, not a dependency.

#	Feature	What It Does	ADHD Problem Solved

1	**Algorithmic Priority feature**	Weighted composite score (urgency decay + importance + energy fit) sorts tasks using Python's Timsort O(n log n). Dashboard updates automatically.	Decision fatigue — the algorithm decides what to do next
2	**ADHD Focus Timer**	Auto-starts from task's estimate_mins value. Three visual states: Focus (green pulse) → Warning (amber, faster pulse) → Alarm (red flash + Web Audio chime). Session logged to MySQL on completion.	Time blindness — external time structure imposed without user initiation

3	**AI Task Breakdown feature**	Gemini 1.5 Flash decomposes a task into 3–5 steps via structured JSON output. Steps are revealed one at a time. User clicks "Done" to advance. "I'm stuck" re-queries Gemini for a simpler first action.	Task initiation paralysis — overwhelming tasks become a single concrete action
4	**Daily Focus View**	Dashboard shows today's tasks sorted by priority score, with a live completion counter. Task completion ticks from any view without page reload.	Routine and structure — one clear view of what matters today

MVP target — 3rd July 2026 (Interim Report): Features 1, 2, and 4 are fully operational. The priority algorithm drives the dashboard. The timer will run on all tasks. The app is demonstrable with zero AI dependency, which means the interim report has real working software to document.

Full product — 2nd August 2026 (Final Submission): Feature 3 (Gemini AI breakdown) is integrated, tested against 20 labelled prompts, and deployed on Railway.

4. Technical Approach

Anchor will use a **Python 3.11 + Flask** backend with **MySQL 8** for persistent data storage, deployed on Railway via GitHub. Server-side rendering uses Jinja2 templates for all standard pages. The client side uses **HTML5, Bootstrap 5, CSS3**, and **JavaScript (ES6+)** — the timer and the AI step-reveal logic are the two meaningful JS components; everything else is plain HTML forms posting to Flask routes.

Algorithm Layer — priority.py

The priority engine lives in a standalone Python module with no Flask dependencies. This makes it independently testable with pytest before a single route is written. The scoring formula is:

```
score(t) = (0.50 × U) + (0.35 × I) + (0.15 × E)
```

Where:

- **U (Urgency)** — exponential decay on days remaining: tasks due today score ≈ 1.0; tasks due in 7 days score ≈ 0.35; no-deadline tasks score 0.50 (neutral). Formula: U = e^(−0.15 × days_remaining). Exponential decay is used because ADHD urgency perception is non-linear — a task due in 1 day feels far more urgent than twice a task due in 2 days [1].
- **I (Importance)** — user-defined priority (1–3) normalised to 0.0–1.0.
- **E (Energy Fit)** — today's self-rated energy level (1–5) normalised to 0.0–1.0. Acts as an accessibility modifier: if energy is low, high-estimate tasks score lower, preventing the app from assigning a 3-hour task to an exhausted user.

Python's built-in sorted() applies Timsort (O(n log n)) to order the task list by score. This is appropriate for the dataset size (5–100 tasks per user) and requires no external library. Big O is documented in code comments and the final report.

ADHD Focus Timer — Client-Side JavaScript

The timer reads estimate_mins from the Jinja2-rendered task card and auto-starts on page load — no user action required. It runs as a **state machine** with three states:

State	Condition	Visual Behaviour	Audio

Focus	> 30% remaining	Green pulsing ring, calm animation	Silent
Warning	10–30% remaining	Amber ring, faster pulse, subtle border glow	Silent
Alarm	0% remaining	Red flash overlay, text "Time's up!"	Soft Web Audio API chime (440Hz sine, 2s fade)

On completion or user click of "Done", JavaScript fetch() POSTs elapsed time and task ID to /session/complete. Flask stores the record in the focus_sessions MySQL table. This data feeds a simple focus streak counter on the dashboard: consecutive days with at least one completed timer session.

AI Breakdown feature — breakdown.py
Gemini 1.5 Flash is called via the google-generativeai Python SDK. The request uses response_mime_type="application/json" to enforce structured output — Gemini returns a JSON array of steps rather than free text, which eliminates parsing errors. Parameters: temperature=0.4, max_output_tokens=200. The system prompt enforces physical verbs ("Open", "Write", "Click") and prohibits advice, encouragement, and meta-commentary.

If the API call fails or times out (8-second limit), a rule-based fallback returns: {"steps": ["Open the file for [task title] and write one sentence."]}. The app never crashes on AI failure.

Technology Stack

Layer	Technology	Rationale
Frontend	HTML5 + Bootstrap 5 + JS ES6+	Server-rendered pages; JS for timer state machine and async AI step fetch only

Backend	Python 3.11 + Flask	Lightweight, appropriate for HDAIML track; clean separation of algorithm and AI modules
Algorithm	priority.py — pure Python	No dependencies; independently testable; Big O documented per function
AI Engine	Gemini 1.5 Flash (google-generativeai)	Structured JSON output mode; free tier 1,500 req/day; used only for breakdown
Database	MySQL 8 + mysql-connector-python	Parameterised queries prevent SQL injection; relational schema with FK constraints
Auth	Werkzeug bcrypt + Flask sessions	Passwords hashed before storage; never logged or stored in plaintext
Deployment	Railway (GitHub CI/CD)	Zero manual server config; Gemini API key stored as environment variable
Testing	pytest + pytest-cov + pytest-benchmark	≥70% line coverage target; Big O empirical benchmarks

5. Data and Resources Required

No external datasets are required. All data is user-generated at runtime: tasks, focus session records, AI breakdown logs, and priority score history. A synthetic seed dataset (14 days of realistic task and

session data) will be generated for demonstration purposes.

Gemini API: Free tier, 1,500 requests per day. Token consumption is reduced by approximately 75% compared to an AI-only architecture because Gemini handles only task breakdown, all sorting, scheduling, and dashboard logic runs in pure Python. Developer holds an active Gemini account.

No GPU, paid compute, or licensed datasets are required. All libraries are open-source and installed via pip. Estimated storage for a 90-day active user: under 500KB , well within Railway's free tier limits.

---

6. Evaluation

Testing covers four categories:

Algorithm Testing (pytest + pytest-benchmark): Unit tests for every function in priority.py — urgency decay computation, importance normalisation, composite score, and Timsort output order. Big O behaviour verified empirically for n = {10, 50, 100, 500} tasks. Target: all sorts complete in under 1ms for n ≤ 500.

AI Breakdown Testing: 20 manually labelled test prompts across four task categories (academic, administrative, creative, technical). Each Gemini response scored for physical verb compliance and actionability on a 1–5 scale. Target: ≥17 of 20 prompts score 4 or higher (85% threshold).

Integration and System Testing: Flask test client tests for all routes. Structured 10-scenario end-to-end walkthrough covering happy path, edge cases (expired deadline, API timeout, empty task list), and security probes (SQL injection attempts, unauthenticated route access).

End-User Testing: Three participants each will use the app for one real task from initiation to completion, with structured feedback collected post-session. Developer (ADHD + GAD) conducts 7 consecutive days of personal daily use from Week 7 — domain-expert feedback that most student projects cannot provide.

SC	Measurable Success Criterion

SC1	Priority algorithm sorts 50 tasks by score in under 1ms; empirical Big O matches O(n log n) for n up to 500 (pytest-benchmark)
SC2	Timer auto-starts within 200ms of page load for any task card with an estimate_mins value
SC3	AI breakdown produces physically-actionable, verb-compliant steps in ≥17 of 20 labelled test prompts (85%)
SC4	All Flask routes respond in under 500ms excluding the Gemini call (pytest-benchmark, 50 sequential requests each)
SC5	Zero unhandled server exceptions across the 10-scenario end-to-end system walkthrough
SC6	App remains fully functional when Gemini API is unavailable — algorithm, timer, and task management operate independently

7. Project Plan and Timeline

Resilience strategy: The algorithm and timer (Features 1, 2, 4) are built and deployed before Gemini is touched. If AI integration is delayed, the interim report demonstrates a fully functional, deployed application. Gemini is an enhancement layer, not a structural dependency.

Phase	Dates	Key Deliverables	Submission

Sprint 0	Wk 1 — Jun 9–13	GitHub repo, MySQL schema (4 tables), Flask setup, user auth (register/ login/logout/ bcrypt)	**SRS due Jun 12**
Sprint 1	Wk 2–3 — Jun 16–27	priority.py with weighted scorer + unit tests, task CRUD, Daily Focus View sorted by algorithm score	—
Sprint 2	Wk 4 — Jun 30–Jul 3	ADHD Focus Timer (3-state JS state machine + Web Audio), focus_sessions table, focus streak counter, Railway MVP deploy	**Interim Report due Jul 3**

Sprint 3	Wk 5–6 — Jul 7–18	breakdown.py Gemini JSON integration, sequential step revelation UI, "I'm stuck" re-query, fallback logic, AI coaching log	—
Sprint 4	Wk 7–8 — Jul 21–Aug 1	pytest suite (≥70% coverage), Big O benchmarks, 20-prompt AI evaluation, 3-participant user testing, security tests, UI polish	—
Final	Wk 9–10 — Aug 1–2	Final technical report, documentation, demo video recording, Railway production deployment	**Final due Aug 2**

Risk Register:

Risk	Probability	Impact	Mitigation	Trigger

Gemini API unavailable during demo	Low	High	App fully functional without AI; local fallback responses pre-cached	API call fails at demo start
Scope creep beyond 10 weeks	High	High	MoSCoW prioritisation; Sprint 3 is the only scope-flexible phase	Week 6 review shows < 50% of Sprint 3 done
Railway free tier downtime	Low	High	Local development environment maintained as backup demo environment	Railway outage confirmed 48h before demo
Python learning curve delays Sprint 1	Medium	Medium	Weeks 1–2 syntax practice runs parallel to environment setup	Auth not functional by end of Week 2

Gemini JSON schema produces malformed output	Medium	Low	Schema validation on Flask side; fallback array returned on parse error	Response fails json.loads()

References

[1] R. A. Barkley, *Executive Functions: What They Are, How They Work, and Why They Evolved*. New York, NY: Guilford Press, 2012.

[2] R. C. Kessler et al., "The prevalence and correlates of adult ADHD in the United States," *American Journal of Psychiatry*, vol. 163, no. 4, pp. 716–723, 2006.

[3] Goblin.tools, "Magic To-Do," [Online]. Available: https://goblin.tools. [Accessed: Jun. 2026].

[4] P. M. Gollwitzer, "Implementation intentions: Strong effects of simple plans," *American Psychologist*, vol. 54, no. 7, pp. 493–503, 1999.

[5] Google, "Gemini API — Structured output," *Google AI for Developers*, 2024. [Online]. Available: https://ai.google.dev. [Accessed: Jun. 2026].

[6] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.

[7] D. Eisenhower (adapted S. R. Covey), *The 7 Habits of Highly Effective People*. New York, NY: Simon & Schuster, 1989.